

課題ガイド(高校生向け)

このゲーム「海賊の秘宝」は、レールを流れてくる玉の列に下の大砲から玉を撃ち込み、**同じ色を3つ以上そろえて消す**タイプのゲームです。この課題集では、実際に動いているゲームのコードを少しずつ読み、書き換えながら、JavaScript とゲーム作りの考え方を学びます。

ゲームは**最初から完成して動く状態**です。壊れるのを恐れず数値やコードをいじってみましょう。おかしくなったら、編集したファイルで Ctrl+Z(元に戻す)を押せば戻せます。

0. 準備

1. **起動**: `index.html` をダブルクリックしてブラウザで開くだけ(ビルドやインストールは不要)。
2. **編集**: フォルダごと VSCode で開く。ファイルを保存したら、ブラウザを**再読み込み(F5)**すると反映される。
3. **コンソール**: ブラウザで **F12** キーを押すと開発者ツールが開く。「Console」タブに**赤いエラー**が出ていたら、書き間違い(カンマ忘れ・カッコ忘れなど)のヒントが書いてある。
4. **課題の場所の探し方**: VSCode の全文検索(**Ctrl+Shift+F**)で **【課題** を検索すると、編集ポイントが全部一覧できる。
5. **便利な起動オプション**: `index.html?level=5` のように URL の後ろに `?level=数字` を付けると、そのステージから始められる(動作確認に便利)。

⚠ 行番号について このガイドには「〇〇行目あたり」という目安も書いてありますが、コードを書き足すと行番号はずれます。**必ず検索(Ctrl+Shift+F)や関数名・変数名で探すクセをつけましょう。**

各課題は、次の流れで書いてあります。上から順に読めば進められます。

1. この課題でやること → 2. 開くファイル → 3. コードの探し方 → 4. いまのコードの意味 → 5. 自分で変更する → 6. 動かして確認する

課題1: 難易度をデザインする

1-1. この課題でやること

このゲームには「みならい海賊(Easy)」から「深海の悪魔(HardCore)」まで4つの難易度があり、**タイトル画面・ゲームオーバー画面・クリア画面**でボタン(または 1~4 キー)で選べます。プレイの途中では変わりません。

難易度の「手ごわさ」は、1つの表の**数値**で決まっています。この課題では、その数値の意味を理解したうえで、**値を変える → 遊ぶ → 比べる**を繰り返して、自分が「ちょうどいい」と思う難易度バランスを設計します。**正解の数値はありません。**

1-2. 開くファイル

`js/config.js` を開きます。ゲームの設定値(数値)だけを集めたファイルです。

1-3. コードの探し方

Ctrl+Shift+F で **TODO【課題1】** を検索すると、大きなコメントの下に **PP.DIFFICULTY = {** で始まる表が見つかります(参考: 46行目あたり)。

```
PP.DIFFICULTY = {
  easy:    { name: "みならい海賊", entryMult: 0.85, holeMult: 0.75, curveMult:
1.15, ... },
  normal:  { name: "一人前の海賊", entryMult: 1.00, holeMult: 1.00, curveMult:
1.00, ... },
  hard:    { name: "海賊船長",      entryMult: 1.10, holeMult: 1.12, curveMult:
0.95, ... },
  hardcore: { name: "深海の悪魔",   entryMult: 1.20, holeMult: 1.13, curveMult:
0.90, ... }
};
```

(実物は1行がもっと長く、この後ろに **timeMult** などが続きます)

💡 **ミニ知識:** オブジェクト { 名前: 値, 名前: 値, ... } という形を JavaScript では**オブジェクト**と呼びます。「名前で引ける表」だと思えばOK。**PP.DIFFICULTY** は「難易度の名前 → その設定」の表で、中の **easy: { ... }** の **{ ... }** もまたオブジェクト(easy の設定一式)です。値のあいだの**カンマ** , を消すとエラーになるので注意。

1-4. いまのコードの意味

パラメーター一覧(表の読み方)

項目	意味	難しくするには
name	画面に出る難易度の名前	—
entryMult	洞窟(スタート)付近の速度の倍率(序盤の圧)	大きくする
holeMult	樽(ゴール)直前の速度の倍率(終盤のプレッシャー。一番効く)	大きくする
curveMult	減速カーブの指数の倍率(下で説明)	小さく する(奥まで速いまま)
timeMult	各ステージで耐える秒数の倍率	大きくする(長期戦)
colorAdd	玉の色数への加算(+1 だと1色早く増える)	大きくする
colorMin	最低色数(全難易度ともステージ1から4色)	大きくする
colorMax	その難易度で使う色数の上限(6=黒以外、7=黒込み。課題2で登場)	大きくする
barrelBonus	樽が呑み込める玉数への加算(基本4個。easy は +2 で6個)	小さくする
useLives	ライフ制(課題5)を使うか。hardcore だけ false	—
bgm	その難易度のプレイ中BGM(課題4で差し替える)	—

1.00 が「基準(Normal)」で、**1.20** なら基準の1.2倍、**0.85** なら0.85倍という**倍率**です。

玉の速度はどう決まる?(いちばん大事なところ)

チェーンの速度は、`js/chain.js` の `speedAt()` という関数で毎フレーム計算されています。課題1で書き換えるのは `config.js` だけですが、この関数を読むと `Mult` の意味がはっきり分かります。Ctrl+Shift+F で `function speedAt` を検索してみましょう。中心はこの5行です。

```
var entry = sp.entry * df.entryMult; // 洞窟付近の速度 × 難易度の倍率
var hole = sp.hole * df.holeMult; // 樽の直前の速度 × 難易度の倍率
var curve = sp.curve * df.curveMult; // 減速カーブの指数 × 難易度の倍率
var t = Math.max(0, Math.min(1, d / lane.rail.holeD));
var v = hole + (entry - hole) * Math.pow(1 - t, curve);
```

1行ずつ読み解きます。

- `sp` は基本の速度設定(`js/config.js` の `PP.SPEED`)。ステージ1では `entry: 800`(洞窟付近 800px/秒)、`hole: 22`(樽の直前 22px/秒)、`curve: 1.8`。※ コースごとに `speed:` で上書きされることがあります(短いコースは遅めに設定してある)。
- `df` は選択中の難易度の設定(さっきの表の1行)。倍率はここで掛かります。
- `t` は「玉がレールのどこにいるか」を `0~1` で表した数。0=洞窟、1=樽。
- `Math.pow(a, b)` は「a の b 乗」を計算する命令。`Math.pow(1 - t, curve)` は「洞窟では1、樽に近づくほど0に近づく数」になります。
- つまり最後の式は「樽の速度 `hole` を土台に、洞窟に近いほど `entry` へ近づける」計算です。

数値で確かめてみましょう(Normal・ステージ1の場合)。

位置 <code>t</code>	<code>Math.pow(1-t, 1.8)</code>	速度 <code>v</code>
0.0(洞窟)	1.00	800 px/秒
0.5(中間)	約0.29	約247 px/秒
0.9(樽の手前)	約0.016	約34 px/秒
1.0(樽)	0	22 px/秒

半分進んだ時点で速度は1/3以下。最初に一気に減速して、あとはじわじわ遅くなる曲線です(一定ペースで減る「直線」ではなく、指数関数のカーブ)。指数 `curve` を大きくすると早めに減速して優しくなり、小さくすると樽の近くまで速いまま=危険。表の `curveMult` が「小さくするほど難しい」のはこのためです。

`speedAt()` はこの後、さらに3つの掛け算をします。

```
v *= 1 + (g.level - 1) * sp.levelStep; // レベルで底上げ
v *= 1 + sp.rollout * g.rolloutBoost; // 開始直後のなだれ込み
if (g.effects.slow > 0) v *= 0.5; // 🐢スロー効果中は半分
```

💡 ミニ知識: `*= v *= 1.2` は `v = v * 1.2` の省略形。「v を1.2倍にして入れ直す」という意味です。

- **levelStep** は **0.06**。ステージが1つ進むごとに**6%ずつ速くなります** (ステージ2で6%増し、ステージ5で24%増し。こちらは一定ペースで増える線形の変化)。
- **rollout** は「ステージ開始直後だけ玉がなだれ込んでくる」演出。**g.rolloutBoost** は最初 1 で、時間とともに指数的に減っていき(**Math.exp** を使った減衰)、数秒で 0 になります。つまり開始直後だけ最大3.2倍速で、すぐ通常速度に落ち着きます。

速度以外のパラメータはどこで効いている?

- **timeMult** — 各ステージは「生存ゲージが尽きるまで耐える」ルール。耐える秒数は **js/main.js**(【課題1】で検索)で「60秒 + ステージごとに10秒、最大90秒」を計算し、それに **timeMult** を掛けています。easy(0.85)は短く済み、hardcore(1.10)は長く耐えます。
- **色数(colorAdd / colorMin / colorMax)** — 色が多いほど「装填した色が今欲しい色じゃない」場面が増えて難しくなります。ステージが進むと1色ずつ増え(**PP.COLORS**)、そこに **colorAdd** を足し、**colorMin** 以上 **colorMax** 以下に収める、という計算を **js/main.js** でしています。
- **barrelBonus** — 樽は基本4個(**PP.BARREL_CAPACITY**)まで玉を呑み込めます。そこへの加算です。easy は +2 で6個(余裕)、hardcore は -1 で3個(即死級)。

💡 **ポイント: 波の間隔や玉の数は難易度で触らない** 玉を補給する波の間隔(**PP.WAVE_INTERVAL**)や1波の玉数(**PP.WAVE_SIZE**)を難易度で変える手もありますが、テンポがだれて「消す気持ちよさ」を削ぐため、このゲームでは**速度プロファイル (entry / hole / curve)**を難易度の本体にする設計です。

1-5. 実験して、自分のバランスを設計する

いきなり4つ全部を変えず、まず **normal** を**実験台**にして1個ずつ効きを確かめます。1回の実験は「値を変える → 保存 → F5 → ステージ1を1〜2分遊ぶ → メモ」で十分です。

- **実験A: holeMult(終盤の圧)** — normal の **holeMult** を **1.00 → 1.50 → 2.00** と変えて 3回遊び比べる。樽の手前での「待ってくれる感じ」がどう消えていくか?
- **実験B: curveMult(速さを保つ範囲)** — **1.00 → 0.85 → 1.40** で比較。0.85 は「奥まで速い」、1.40 は「早めにゆっくり」。holeMult との効き方の違いは?
- **実験C: entryMult(序盤の圧)** — **1.00 → 1.30** で比較。序盤の忙しさだけが変わるはず。

🔍 **確認してみよう:** 実験が終わったら normal を **1.00 / 1.00 / 1.00** に戻し、メモをもとに **easy** と **hardcore** を**自分の数値で設計**しましょう。考える問いはこの2つ。

- 初めて遊ぶ人に「考える時間」をあげるには、どの値をどっちに動かす?
- 上級者を本気にさせるには、速度・色数・樽の余裕のどれで攻める?

⚠ **注意** 数値のあとの **,**(カンマ)や **{ }** を消すと JavaScript のエラーになります。画面が真っ暗になったら F12 のコンソールを見て、Ctrl+Z で戻しましょう。

発展: **PP.DIFFICULTY** に5つ目の難易度(例: **nightmare**)を追加してみましょう。すぐ下の **PP.DIFFICULTY_ORDER** (ボタンの並び順の配列)にも名前を足すと、ボタンが5つに増えます。

1-6. 動かして確認する

1. 保存 → ブラウザを再読み込み(F5)。
2. タイトル画面で難易度を選んで(1〜4キーが速い)プレイし、狙いどおりの体感が確かめる。
3. うまいかないとき:

- 変化を感じない → 保存し忘れ / F5 し忘れ / 選んだ難易度が実験した難易度と違う、をまず疑う。
- 画面が出ない・止まる → F12 のコンソールの赤いエラーを読む(たいていカンマかカッコ)。
- プレイ中に難易度が変わらないのは仕様です(選べるのはタイトル・ゲームオーバー・クリア画面だけ)。

課題2: 最後の「黒玉」を完成させる

2-1. この課題でやること

玉の色は7色ぶん用意されていますが、7色目の「黒」は色が未完成(仮のグレー)で、まだ封印中です。この課題では、黒玉の4つの色を自分でデザインし、封印を解いてゲームに登場させます。

2-2. 開くファイル

`js/config.js` を開きます(課題1と同じファイルの少し下)。

2-3. コードの探し方

Ctrl+Shift+F で **TODO** 【課題2】を検索すると、2か所見つかります。

- **TODO** 【課題2-A】 — `PP.PALETTE` という配列の最後(参考: 297行目あたりから)
- **TODO** 【課題2-B】 — `PP.COLORS = { base: 3, perLevel: 1, max: 6, levelStep: 1 };` (参考: 323行目あたり)

2-4. いまのコードの意味

`PP.PALETTE` は玉の色の一覧です。

```
PP.PALETTE = [  
  { light: "#ff6a4a", main: "#e01810", dark: "#4d0708", edge: "#1c0202" }, // 赤  
  ...(青・緑・黄・紫・骨白)...  
  { light: "#888888", main: "#666666", dark: "#444444", edge: "#222222" } // 黒  
  (未完成)  
];
```

💡 **ミニ知識: 配列** `[ 要素, 要素, ... ]` という形が配列(順番のあるリスト)です。 `PP.PALETTE` は「色オブジェクトが7個並んだ配列」で、ゲームは「色番号〇番」でここから引きます。

1つの玉は4色の塗り分けでできています。

- `light` = 左上の照り返し(ハイライト) / `main` = 玉の本体色
- `dark` = 下側の陰 / `edge` = 輪郭

色は `"#RRGGBB"` という16進数の色コードで書きます。VSCode ではコード左の色の四角をクリックするとカラーピッカーが開くので、数値を覚える必要はありません。

そして色数の決まり方。ステージが進むと1色ずつ増えますが(Stage1・2=4色 → Stage3=5色 → Stage4以降=6色)、上限が2段階構えになっています。

- `PP.COLORS` の `max: 6` — ゲーム全体の上限(ここが封印)
- 難易度ごとの `colorMax`(課題1の表)— Easy/Normal は 6、Hard/HardCore は 7

実際の色数は `js/main.js` で「両方の小さい方」を採っています。だから `max` を 7 にしても、**黒が出るのは Hard / HardCore だけ**。Easy / Normal は最後まで黒抜きで戦う設計です。

2-5. 自分で変更する

1. 【課題2-B】を先にやるのがおすすめ: `PP.COLORS` の `max: 6` を `max: 7` に変える。→ 仮のグレー玉が出るようになるので、色を調整しながら見比べられる。
2. 【課題2-A】: `PP.PALETTE` 最後の黒(未完成)の4つの色コードを自分でデザインする。
 - コツはコード中のコメントにも書いてあります: 本体(`main`)を真っ黒にすると夜の海に沈んで見えなくなる。照り返し(`light`)を「月明かりの青灰」(青みがかった明るいグレー)にして輪郭を浮かせる。すぐ上の「骨白」の4色の組み方がお手本。

⚠ 注意 色コードを囲む " (ダブルクォート)や、後ろの , を消すとエラーになります。 # を消すと色として認識されません。

2-6. 動かして確認する

1. タイトルで難易度 **Hard**(3キー)を選び、`index.html?level=5` で最終ステージへ直行。
2. 黒玉が流れてくる・撃てる・3個そろって消える、を確認できたら完成。
3. 🔍 **確認してみよう**: 難易度 Easy / Normal では `max: 7` のままでも黒が出ないことも確認しましょう(それが正しい挙動。理由は 2-4 の「2段構えの上限」)。
4. 黒玉が背景に溶けて見つらいときは、`light` をもっと明るくして再挑戦。

課題3: 自作コースを JSON でゲームに登録する

3-1. この課題でやること

ゲーム内のコースエディタで作ったコースを **JSON** というデータ形式で書き出し、それをゲームに読み込ませて、**自分のコースを「ステージ6」として登録**します。

3-2. 開くファイル

`js/my-course.js` を開きます。60行ほどの短いファイルなので、**まず全部読んでみましょう**。冒頭のコメントに手順と JSON の読み方がまとまっています。

3-3. コードの探し方

ファイル中央の2か所だけ編集します。

- `var MY_COURSE_JSON = null;` — JSON を貼り付ける場所(「手順4」のコメントの下)
- `TODO【課題3】` — 登録のコードを**2行**書く場所

3-4. いまのコードの意味

```
var MY_COURSE_JSON = null;
// ...
```



```
if (!MY_COURSE_JSON) return;
```

- `null` は「まだ何も入っていない」を表す特別な値です。
- `if (!MY_COURSE_JSON) return;` は「何も貼られていなければ、ここで終了」。だから今はこのファイルがあっても、ゲームは通常どおり5ステージで動いています。
- そのあとの `try { ... } catch (e) { ... }` は「中でエラーが起きてもゲーム自体は 起動させる」ための保険です。失敗すると F12 のコンソールに「【課題3】 コースを登録できませんでした」と出ます。

💡 **ミニ知識: JSON と JavaScript オブジェクト** JSON は「JavaScript のオブジェクトの書き方 `{ ... }` を、データの受け渡し用に決めた形式」です。だからエディタが保存した JSON ファイルの中身は、そのまま JavaScript のコードとして貼れます。

JSON の中身で特に大事なキーは3つ。

- `ctrl` — レールの形を決める点 `[x, y]` の並び。最初の点が洞窟(入口)、最後の点が樽(ゴール)
- `sharp` — `true` なら直角に曲がる道、`false` ならなめらかな曲線
- `lanes` — 道(レーン)の配列。トンネル(`tunnels`)や橋(`raised`)の区間もここに入る

3-5. 自分で変更する(手順)

1. ゲーム画面で **E キー**(または「✖ コース作成」ボタン)でエディタを開く。
2. コースを作り、「↓ **ファイル保存**」で JSON ファイルをパソコンに保存する。
3. その JSON ファイルをメモ帳か VSCode で開き、**中身を全部コピー**する。
4. `MY_COURSE_JSON = null;` の `null` だけを消して、コピーした中身を貼り付ける。行末の `;` は残すこと。
5. **TODO 【課題3】** の場所に、登録のコードを**2行**書く。ヒントはコメントにあります。
 - 1行目: `PP.courseAPI.fromJSON(MY_COURSE_JSON)` は、貼り付けた JSON をゲームが使える「コースオブジェクト」に**変換して返す**関数。返ってきたものを `var` で変数に受け取ろう。
 - 2行目: その変数の `.register()` を呼ぶと `PP.COURSES` に追加され、新しいステージになる。
 - 2行をどうつなぐかは自分で組み立ててみましょう(各行の最後に `;` を忘れずに)。

3-6. 動かして確認する

1. 保存 → F5 → `index.html?level=6` で自分のコースが始まれば完成。ステージ5をクリアした後にも自動で登場します。
2. うまいかないとき:
 - **何も起きない** → `null` の消し忘れ / 貼り付けた JSON の後ろの `;` が消えていないか確認。
 - **コンソールに「【課題3】 コースを登録できませんでした」** → 貼り付けが欠けている (JSONの先頭か末尾が欠けやすい)。もう一度全選択してコピーし直す。
 - **赤いエラーが出る** → 2行のコードのつづり(`courseAPI` の大文字小文字など)を確認。
3. 🔍 **確認してみよう:** `.register()` の代わりに `.play()` にすると、起動した瞬間に そのコースで遊べます(動作確認に便利。終わったら `.register()` に戻そう)。

課題4: SE・エフェクト・BGM のカスタマイズ

4-1. この課題でやること

音(BGM・効果音)と演出を自分好みに差し替えます。メイン課題は「**難易度ごとに BGM を変える**」、サブ課題は「**カラーボムの効果音を差し替える**」です。

4-2. 開くファイル

`js/config.js`(BGMの指定)、`js/audio.js`(音の一覧)、`js/fx.js`(見た目の演出)。編集ポイントはすべて `Ctrl+Shift+F` で **TODO【課題4】** を検索すると見つかります。

4-3～4-4. メイン課題: 難易度ごとに BGM を組み込む

いまは4難易度とも**同じ曲**(`BGM/Game_music.mp3`)が流れています。仕組みはこうです。

1. `js/config.js` の `PP.DIFFICULTY`(課題1の表)の各行に `bgm: "BGM/Game_music.mp3"` がある。
2. ゲーム開始時に `js/audio.js` の `gameStart()` という関数が、選択中の難易度の設定から `PP.diff().bgm` でこのパスを読み取り、その曲を再生する。
3. 危機のとき(`BGM/Denger.mp3`)とゲームオーバー(`BGM/gameover_BGM.mp3`)への切り替えは 全難易度共通。

やること:

1. 好きな mp3 を `BGM/` フォルダに入れる(例: `BGM/hardcore.mp3`)。
2. `PP.DIFFICULTY` の **その難易度の行だけ** `bgm:` のパスを差し替える。
3. 保存 → `F5` → その難易度で開始して、曲が変わったら成功。

🔍 **確認してみよう:** 他の難易度で始めると元の曲のまま、が正しい状態です。

4-5. サブ課題: カラーボムの SE を差し替える

カラーボム(💣)をキャッチすると同じ色の玉が全部消えて音が鳴りますが、この音はいま **別の場面の音を借りているだけ**です。専用の音にしてあげましょう。

1. 好きな mp3 を `SE/` フォルダに入れる(例: `SE/colorbomb.mp3`)。
2. `js/audio.js` で **カラーボム** を検索し、**TODO【課題4】** コメントの下のある行を見つける。

```
var seColorBomb = sfx("SE/broken_treasure.mp3", 0.8);
```

3. ファイル名の部分を自分の mp3 に差し替える。

💡 **ミニ知識: 関数呼び出しと引数** `sfx("SE/ファイル名.mp3", 0.8)` は「`sfx` という関数に、**ファイルパス**と**音量**の2つの材料(引数)を渡して呼び出す」という書き方です。音量は 0(無音)～ 1(最大)。結果(再生できる音)を `var seColorBomb = ...` で**変数に入れておき**、カラーボム発動の瞬間に鳴らしています。ファイル名は `"` で囲んだまま変えること。

4. 保存 → `F5` → カラーボムをキャッチして音が変わったら成功。音量(第2引数)も調整してみよう。
5. 音が鳴らないとき: `F12` のコンソールにファイルが見つからないエラー(404)が出ていないか確認。ファイル名の**大文字小文字・拡張子**の打ち間違いが定番です。

4-6. そのほかのカスタマイズ(自由研究)

下の表の場所はどれも **TODO【課題4】** か、表内の検索ワードで見つかります。1か所変える → F5 で確かめる、を繰り返しましょう。

変えたいもの	場所(検索ワード)	変えると
効果音の音量・差し替え	<code>js/audio.js</code> の <code>sfx(</code> の並び	その場面の音・大きさが変わる
BGM 全体の音量	<code>js/audio.js</code> の <code>BGM_VOL</code>	全曲の音量(0~1)
危機・ゲームオーバーの曲	<code>js/audio.js</code> の <code>track("BGM/</code>	場面の曲が変わる
危機警報の音量	<code>js/audio.js</code> の <code>CRISIS_VOL</code>	樽ピンチ時の警報の迫力
パワーアップの出やすさ	<code>js/config.js</code> の <code>PP.POWERUPS</code> の <code>w</code>	種類ごとの出現率(相対値。大きいほど出やすい)
アイテムのドロップ率	<code>js/config.js</code> の <code>PP.ITEMS</code> の <code>dropChance</code>	落ちる頻度
玉が消える破片の数	<code>js/fx.js</code> の <code>for (var i = 0; i < 7; i++)</code>	消えたときの派手さ(30以下推奨)
爆弾の威力(範囲)	<code>js/config.js</code> の <code>PP.BOMB_RADIUS</code>	爆風で消える玉の数と見た目
危機演出の強さ	<code>js/config.js</code> の <code>PP.CRISIS</code>	画面の赤さ・揺れ・心拍

課題5: コインを集めてライフ回復(1発ゲームオーバーの緩和)

5-1. この課題でやること

出航時にライフを2つ(右上の HUD に ♥ で表示)持っていて、樽が溢れてもライフが残っていれば**そのステージの最初から**やり直せます。一方、玉を消すと落ちてくる コイン 🪙 は、キャッチすると枚数は増えるのに**まだ何も起きません**。

この課題は、同じ「ライフのしくみ」を3つの向き合い方で学ぶ3部構成です。

パート	やること	スタイル
課題5-1	コインが5枚たまったらライフが1回復する	自分で書く (中身が空の関数を完成させる)
課題5-2	樽あふれ時の「ライフで復帰 or ゲームオーバー」の分岐	模範解答を読み解く (答え入りのコードに問いが付いている)
課題5-3	リトライの画面切り替えの「間」	模範解答を調整する (数値を変えて心地よさを探す)

「書く・読む・調整する」はどれもプログラミングの大事な練習です。特に 5-2 と 5-3 は、**動いている完成コードを壊して・直して理解する**という、現場でもよく使う学び方をします。

💡 ミニ知識: この課題で使う書き方

- `if (条件) { A } else { B }` — 条件が成り立てば A、だめなら B を実行。
- `g.coins >= 5` — 「以上」の比較。成り立つ/成り立たないの答えになる。
- `g.lives -= 1` は「1減らす」、`g.lives += 1` は「1増やす」(`g.lives = g.lives - 1` の省略形)。

5-2. 課題5-1: コイン → ライフ回復(js/powerups.js)

Ctrl+Shift+F で **TODO【課題5-1】** を検索すると、`checkCoinLife()` という**中身が空の関数**が見つかります。この関数は、コインをキャッチするたびに(`collect()`の中から)すでに呼ばれています。呼ばれる仕組みはできていて、**中身だけが未完成**、という状態です。

関数の先頭にはすでに1行あります。

```
if (PP.diff().useLives === false) return;
```

難易度「深海の悪魔」(hardcore)は `useLives: false` なので、ここで処理を打ち切っています (課題1の表とつながっているのが分かりますか?)。

この下を書く処理を、日本語で組み立てると:

1. もし コイン(`g.coins`)が `PP.LIFE.coinsPerLife` 枚(初期値5)以上たまっていて、
2. かつ ライフ(`g.lives`)が上限 `PP.LIFE.maxLives` より少なければ、
3. コインを `coinsPerLife` 枚ぶん減らして、ライフを1増やす。

これを JavaScript にするのが課題です。コードのコメントにもヒントがあります。回復できたときに次の1行を入れると、画面に「♥ ライフ +1!」と出て気持ちいいです。

```
PP.fx.floatText("♥ ライフ +1!", PP.W / 2, 96, "#ff5d8f", 24);
```

🔗 **考えてみよう:** ライフが上限のときに拾ったコインをどうするか(貯めておく? 捨てる?)で if の書き方が変わります。どちらにするかは自分で決めてOK。

5-3. 課題5-2: 樽あふれの分岐を読み解く(js/main.js)

Ctrl+Shift+F で **【課題5-2】** を検索します。すぐ上に、こんな判定があります。

```
if (lane.balls.length > 0 &&
    lane.balls[0].d >= lane.rail.holeD + PP.D * PP.barrelCap()) {
    deadLane = lane; return false;
}
```

- `lane.balls[0]` は**先頭の玉**(樽にいちばん近い玉)。`.d` はレール上の位置。
- 「先頭の玉の位置が、樽の口(`holeD`)+ 許容個数ぶん(`PP.D × PP.barrelCap()`)を超えたか」を調べています。超えたレーンがあれば `deadLane` に入ります。

その下の分岐が今回の主役で、ここには模範解答がすでに入っています。

```
if (PP.diff().useLives !== false && g.lives > 0) {  
  g.lives--;  
  retryLevel();  
} else {  
  gameOver();  
}
```

日本語に訳すと「ライフ制の難易度で、かつライフが残っていれば、ライフを1つ使って `retryLevel()` (そのステージの最初からやり直し)。そうでなければ `gameOver()`」。

読んで分かった気になったら、コードのコメントにある3つの問いを 予想 → 実際に書き換えて試す → 元に戻す (Ctrl+Z) の順で確かめてください。遊んで確かめてから、下の解説を読みましょう。

Q1. `&&` の左右(`useLives` の条件と `g.lives > 0`)を入れ替えても同じ動き?

▶ 試してから開く

Q2. `g.lives > 0` を `g.lives >= 0` にすると、何が壊れる?

▶ 試してから開く

Q3. `g.lives--` を `retryLevel()` の「後」に動かすと、画面のどこがおかしくなる?

▶ 試してから開く

5-4. 課題5-3: リトライの画面切り替えの「間」 (`js/config.js`)

課題5-2 の `retryLevel()` は、最初は `startLevel()` (盤面の組み直し) を呼ぶだけでした。それだと樽が溢れた次のフレームには新しい盤面が出ていて、切り替わりが急すぎます。プレイヤーには「何が起きた? ライフは減った?」を確認する**間(ま)**が必要です。

そこで、いまの `retryLevel()` は切り替えを3拍子に分けています (`js/main.js` で【課題5-3】を検索すると実物が読めます)。

拍	画面	ねらい
① 静止	ミスした盤面のまま時が止まる(揺れ+重い音)	「樽が溢れた」瞬間を見せる
② 暗転	暗くなり「♥ 残りライフ — 同じ海域の最初から」	状況を確認する間を作る
③ 明転	暗転の裏で盤面を組み直してからフェードで再開	組み直しの瞬間を見せない

しくみは「`state` を "`retrying`" にすると `tick` がプレイ処理を素通りする」 + 「`dt` で減るタイマーが切れたら次の拍へ」。課題5-2 の `if/else` や課題6 の `dt` と、同じ部品の組み合わせでできています。

ここでの課題は書くことではなく調整です。`js/config.js` で `TODO` 【課題5-3】を検索すると、各拍の長さが出てきます。

```
PP.RETRY = {
  freeze: 0.5,    // ①ミスした盤面のまま時間が止まる長さ(秒)
  veilTime: 1.2,  // ②暗転して残りライフを見せる長さ(秒)
  fade: 0.45,     // ③明転(フェード)の長さ(秒)
  veil: 0.8       // 暗転の暗さ(0=透明 ~ 1=真っ暗)
};
```

予想 → 変更 → 体感 で試してみましょう。

- `freeze: 0` にすると? → 切り替わりの何が失われるか、体で確かめる。
- `veilTime: 5.0` にすると? → 長すぎる「間」はストレスになることを体感する。
- 「状況が分かる長さ」と「テンポの良さ」のトレードオフの中で、自分のちょうどいい値を決める。
正解の数値はありません(市販のゲームもタイトルごとに違います)。

5-5. 動かして確認する

難易度は **Normal**(hardcore 以外)で確認しましょう。

1. わざと樽を溢れさせる → 画面が一瞬止まって暗転し、「♥ 残りライフ 1 — 同じ海域の最初から」 と出てから、同じステージが最初から始まる(スコアは残っている)。
2. ライフが減った状態でコインを5枚キャッチ → 「♥ ライフ +1!」 が出て、右上の ♥ が回復する。
3. ♥ を使い切ってから溢れさせる → リトライではなく、ゲームオーバー演出になる。

うまくいかないとき:

- コインが5枚たまってもライフが増えない → 5-1 の比較(`>=` の向き)と、難易度が **hardcore** になっていないかを確認。
- Q1~Q3 の実験のあと挙動が変 → 5-2 の `if/else` を元の形に戻し忘れていないか確認。
- F12 のコンソールに赤いエラー → カッコ `{ }` と `;` の対応を見直す。

5-6. 注意と発展

⚠ 注意 難易度「深海の悪魔」(hardcore)は `useLives: false` なので、正しく実装しても ライフ回復は効きません(HUD の ♥ が × 表示になります)。これは仕様です。

発展: 開始時のライフ(`startLives`)、何枚で回復するか(`coinsPerLife`)、上限(`maxLives`)は `js/config.js` の `PP.LIFE` で調整できます。課題1で作った自分の難易度に合う値を考えてみましょう。

課題6: 倍速モードを作ろう(2倍速・0.5倍速)

6-1. この課題でやること

ゲームの中で流れる時間を、速くしたり遅くしたりする機能を作ります。テレビの録画を「2倍速で見る」「スローで見る」のと同じイメージです。

「○倍速」というのは、通常の速さを 1 としたときの倍率のことです。

モード	倍率	意味
通常速度	1.0	いつもどおりの速さ(1倍)
2倍速	?	通常の 2倍 の速さ。玉もタイマーも全部2倍で進む
0.5倍速	?	通常の 半分 の速さ。全部ゆっくりになる(スローモーション)

「?」の値は、この課題の中で自分で考えて入れます。

大事なのは、**玉だけ**を速くするのではないことです。「2倍速」は、玉が動く**速さ**と、タイマーなどの**時間の進み**の両方が2倍になること。玉・発射した弾・残り時間のゲージ・樽の危機演出(画面が赤くなるあれ)まで、**ゲーム全体の時間**が一緒に速くなったり遅くなったりして、はじめて「倍速モード」と呼べます。「速さ」と「時間」がどうつながっているかは6-3で見ます。

この課題を終えると、「**たった1つの数値を変えるだけで、プログラム全体の動きを調整できる**」という感覚がつかめます。

6-2. 開くファイル

2つのファイルを行き来します。

- `js/config.js` — 倍率の値 `timeScale` が置いてある(変数の名前は「time=時間」「scale=倍率」)
- `js/main.js` — 毎フレーム呼ばれる `tick` 関数と、切り替え用の `cycleTimeScale` 関数

Ctrl+Shift+F で **【課題6】** を検索すると、編集ポイントが全部見つかります。 `timeScale` で検索しても同じ場所にたどり着けます。行番号はコードを書き足すとずれるので、必ず検索で探しましょう。

6-3. まず仕組みを読む(STEP 1)

このゲームの「時間」はどう進んでいる?

`js/main.js` の `tick` という関数は、**1秒間に約60回**(パソコンの画面の描き替えのたびに) 呼ばれます。その先頭近くにこの1行があります。

```
var dt = Math.min(e.delta / 1000, 0.05);
```

- `dt` は「**前のフレームから何秒たったか**」という数(delta time の略)。1秒に60回なら、だいたい `0.0167`(約60分の1秒)です。
- `Math.min(..., 0.05)` は「大きくても 0.05 秒まで」という安全装置 (別タブを見ていて久しぶりに呼ばれたとき、時間が一気に進みすぎないように)。

そしてこの `dt` が、`tick` の中でゲームのあらゆる部品に配られていきます。

```
PP.chain.update(dt);           // 玉の列を dt 秒ぶん進める
PP.cannon.updateShots(dt);      // 発射した弾を dt 秒ぶん進める
PP.powerups.update(dt);         // アイテムの落下・効果の残り秒数
...
PP.crisis.update(dt);           // 樽ピンチの演出
```

「速さ」と「時間」の関係

課題1で見たように、玉の速さは「**1秒あたり何ピクセル進むか(px/秒)**」で決めてあります。実際に1フレームで動く距離は、

動く距離 = 速さ(px/秒) × 経過時間 dt(秒)

です。残り時間のゲージも `timeLeft -= dt`(残り秒から dt を引く)で減っていきます。つまりこのゲームでは、**移動もタイマーも演出も、ぜんぶ「dt が何秒か」で進む量が決まる**仕組みになっています。

ここが今回のポイントです。速さの数値(800px/秒 など)を1個ずつ2倍にして回らなくても、**配られる前の dt を1か所で2倍にすれば、ゲーム全体が2倍速になる**のです。

次に `js/config.js` で `timeScale` を検索してください。`PP.game` という「ゲームの今の状態をまとめたオブジェクト」の中に、こう書いてあります。

```
timeScale: 1.0,
```

これが今回の主役、**時間の倍率**です。今は `1.0` = 通常速度。ただし、**この値はまだどこにも使われていません**。使う処理を次のSTEPで自分で書きます。

6-4. 自分で変更する(STEP 2〜3)

STEP 2: dt に倍率を掛ける(js/main.js)

`js/main.js` で `TODO【課題6】` を検索すると、さっきの `var dt = ...` の直後に `TODO` コメントがあります。そこに、**dt に `PP.game.timeScale` を掛ける1行**を書きます。

💡 **ミニ知識: 掛け算で「倍」を作る** JavaScript の掛け算は `*` です。

- `speed * 2` → 元の速さに 2 を掛ける = **2倍**になる
- `speed * 0.5` → 0.5 を掛ける = **半分**になる(1 より小さい数を掛けると減る)
- `speed * 1` → 1 を掛けても**変わらない**(だから 1.0 が「通常速度」)

また `dt = dt * 2;` のように「自分に掛けて自分に戻す」書き方は、`dt *= 2;` と短く書けます(`*` は「掛けて代入」)。どちらで書いてもOK。

書けたら保存して F5。この時点では見た目は何も変わりません。`timeScale` が `1.0`(=1を掛けるだけ)だからです。エラーが出ていないことだけ F12 のコンソールで確認しましょう。

STEP 3: 倍率を直接書き換えて実験する(js/config.js)

js/config.js の `timeScale: 1.0`, の数値を書き換えて、保存 → F5 で遊び比べます。

1. まず **2倍速** にしてみましょう。

ヒント: 通常を `1.0` とすると、「通常の2倍の速さ」にするにはどの値を掛ければよいでしょうか? `speed * 2` の考え方がそのまま使えます。

2. 遊んでみて、**何が変わったか観察**します。
 - 玉の列は速い? 発射した弾は? 残り時間ゲージの減りは? 樽ピンチの心臓の鼓動は?
 - **全部が一緒に**速くなっていれば成功です。
3. 次に **0.5倍速**(通常の半分)にして、同じことを観察します。ぬるっとしたスローモーションの中で、じっくり狙えるはずです。
4. 終わったら `1.0` に戻しておきます。

実は STEP 2 の掛け算が書けた時点で、**タイトル画面の「時間の流れをえらぶ」ボタン** (難易度ボタンの下の $\times 0.5 / \times 1 / \times 2$) も使えるようになっています。ファイルを 書き換えなくても、出航前にクリックで選べるので、遊び比べはこちらが手軽です。選んだ値は STEP 1 で見た `PP.game.timeScale` にそのまま入ります (逆に言うと、STEP 2 の掛け算を書くまでは、ボタンで選んでも**何も変わりません**。`timeScale` は「使われてはじめて意味を持つ数値」だからです)。

 **考えてみよう:** `timeScale` を `0` にしたら何が起こるでしょう? 予想してから試してみてください(dt に `0` を掛けると「経過時間0秒」= 時間が止まる...?)。試したら `1.0` に戻すのを忘れずに。

 **観察ポイント: 変わらないものもある** 玉が消えるときの破片などの**見た目の演出の一部**は、CreateJS の Tween という 別の仕組み(実際の時計のミリ秒)で動いているため、倍速にしても速さが変わりません。ゲームの進行には影響しないのでこの課題ではそのままにしますが、「同じゲームの中に、dt で進む時間と、実際の時計で進む時間の2種類がある」ことに気づけたら鋭いです。

6-5. プレイ中も切り替えられるようにする(STEP 4)

タイトル画面のボタンで「出航前に選ぶ」ことはできるようになりました。仕上げに、**プレイの真っ最中でも切り替えられるように**します。

プレイ中、画面右上の **II** (ポーズ)ボタンの**左隣**に、 **$\times 1$** と書かれたボタンがすでに置いてあります(S キーでも同じ動作)。押すと js/main.js の `cycleTimeScale` という関数が呼ばれる配線まではできていて、**中身が未完成**です。TODO【課題6】で検索して、`cycleTimeScale` の中を完成させましょう。

日本語で組み立てると:

1. もし 今の `g.timeScale` が通常速度(`1.0`)なら → 2倍速の値を代入する
2. そうでなくて、今が2倍速なら → 0.5倍速の値を代入する
3. そのどちらでもなければ(=0.5倍速なら) → 通常速度 `1.0` に戻す

これで、押すたびに **通常 → 2倍速 → 0.5倍速 → 通常 → ...** とぐるぐる一巡します。

 **ミニ知識:** `if / else if / else` 「今の値と等しいか」は `===` で調べます(= 1個は「代入」なので別物!)

```
if (g.timeScale === 1) {  
    // 通常速度のとき にすること  
} else if (...) {  
    ...  
} else {  
    ...  
}
```

課題5で書いた if/else の3択版です。

ボタンの表示($\times 1/\times 2/\times 0.5$)は `hud.js` が `PP.game.timeScale` の値を見て自動で 描き替えるので、自分で書くのは値を切り替える部分だけでOKです。

なぜ「1つの変数」にまとめるのか

2倍速にする方法として、「玉の速度 800 を 1600 に、弾の速度 1400 を 2800 に、タイマーも...」と速さの数値を1個ずつ全部書き換える手もあります。でも、

- 書き換える場所が何十か所もあって、1か所でも忘れると「玉だけ速い」半端なゲームになる
- 0.5倍速も作るなら、また全部やり直し

今回のように `timeScale` という1つの変数にまとめておけば、変更は1か所で済み、`2.0` でも `0.5` でも `1.5` でも自由自在です。「同じ意味の数値をあちこちに直接書かず、1つの変数にまとめる」— これはプログラムが大きくなるほど効いてくる考え方です。

6-6. 動かして確認する

1. `config.js` の `timeScale` が `1.0` の状態で F5 → いつもどおりの速さで遊ぶ。
2. タイトル画面で $\times 2$ を選ぶ(金縁で光る)→ 出航すると、玉・弾・ゲージ・危機演出が **全部速い**。 $\times 0.5$ を選び直すとスローで始まる。
3. プレイ中に速度ボタン(II の左)か S キーを押す → 画面に「時間の流れ $\times 2$ 」と出て、その場で速くなる。ボタンの表示も $\times 2$ になる。
4. もう1回押す → $\times 0.5$ でスローモーションになる。もう1回で $\times 1$ に戻る。
5. 2倍速のまま P キーでポーズ → 完全に止まる。再開すると2倍速のまま続く。
6. F12 のコンソールに赤いエラーが出ていない。

うまくいかないとき:

- 値を変えても・タイトルのボタンで選んでも速さが変わらない → STEP 2 の「dt に掛ける1行」が書けていない(`timeScale` は使わなければただの数値です)。
- ボタンを押しても「 $\times 1$ 」のまま → `cycleTimeScale` の中身が未完成。
- 1回目は変わるが2回目から動かない → if/else の条件を見直す (= と === を間違えていないか、3つの場合を全部書いたか)。
- 2倍速で弾が玉をすり抜ける気がする → それを防ぐ補助処理(`cannon.js` の `updateShots`)が入っているので、本来は起きません。コンソールのエラーを確認。

発展: 3倍速(`3.0`)や4倍速も試してみましょう。どこまで上げるとゲームに なりませんか? 逆に `0.1` の超スローは? 「ちょうど遊べる範囲」を探るのも設計のうちです。

全課題共通: 完成チェックリスト

- ☐ 課題1: 難易度ごとに体感が明確に変わる、自分の設計にした
- ☐ 課題2: 難易度 Hard の ?level=5 で黒玉が出て、消せる
- ☐ 課題3: ?level=6 で自作コースが遊べる
- ☐ 課題4: 難易度ごとに BGM を変えた(+音やエフェクトも自分好みに)
- ☐ 課題5: コイン5枚 → ♥+1 が動き、樽あふれでは静止→暗転をはさんで復帰する。5-2 の Q1～Q3 を実験した
- ☐ 課題6: ボタン/Sキーで 通常 → 2倍速 → 0.5倍速 が一巡し、ゲーム全体の時間が変わる
- ☐ F12 のコンソールに赤いエラーが出ていない